

Preclass Learning: Functions and Modularity (ICA 9)

1. Custom Function Structure

A function is a named block of code designed to perform a specific task.

- **Syntax:** `function_name <- function(argument_1, argument_2, ...) { code_body; return(output) }`
- **Arguments:** The inputs the function takes. Should be clearly documented.
- **Body:** The code that manipulates the inputs.
- **Return Value:** The value the function outputs. If `return()` is omitted, the last evaluated line of code is returned.

2. Principles of Modularity

- **DRY (Don't Repeat Yourself):** If you copy-paste a block of code more than twice, it should be turned into a function.
- **Single Responsibility:** A function should do *one thing* and do it well. Avoid functions that perform multiple unrelated tasks.
- **Readability:** Use descriptive names for both the function and its arguments.
- **Side Effects:** Minimize external effects (like printing messages or saving files) unless that is the function's explicit purpose (walk function style).

3. Scoping

- **Lexical Scoping:** R determines the value of a variable based on where the function was **defined**, not where it was called. This makes functions predictable and prevents accidental interaction with global variables.
- **Environment:** The environment in which the function was defined is checked first when looking up variable values.